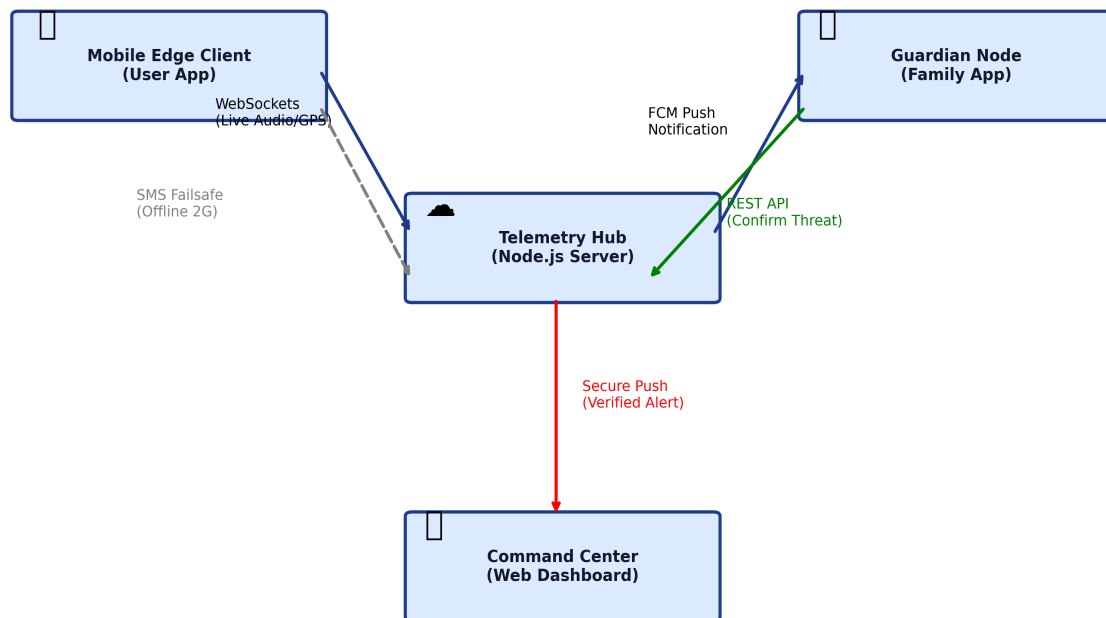


RAHBAR: System Architecture & Workflow Specification

Project: RAHBAR – the Intelligent Transit Security and SOS System

Professional Event-Driven Architecture Flowchart



Architecture Paradigm: Event-Driven Client-Server Architecture with Edge Computing

1. Architectural Overview: The "Broadcaster and Listener" Model

RAHBAR operates on an Event-Driven System using a "Publish and Subscribe" (Pub/Sub) methodology.

In traditional systems, modules call each other directly like a phone call. If one module is busy or offline, the entire system crashes waiting for a response. RAHBAR eliminates this risk by using a Broadcaster and Listener concept.

The Analogy: Imagine a radio station. The radio tower (Broadcaster) simply sends music into the air; it does not care who is listening. Your car radio (Listener) is tuned to that specific frequency to catch the music and play it. In RAHBAR, one module "broadcasts" an emergency event, and the next module "listens" for that specific event to activate its job.

2. Module Connectivity: How the Bridges Work (Step-by-Step)

To ensure seamless communication without overlapping or crashing, RAHBAR uses specific technical "bridges" to connect its core modules during an emergency.

Step 1: Mobile Phone Connecting to the Server (The Telemetry Hub)

The Bridge: WebSockets (Socket.io)

How it Works: Think of a standard API like sending a letter in the mail—you have to wait for a reply. Think of a WebSocket like an open walkie-talkie. As soon as the AI on the phone detects a threat, it uses this continuously open walkie-talkie line to instantly stream the emergency audio and live GPS to the server in milliseconds.

Step 2: Mobile Phone Connecting to the SMS Gateway (The Offline Failsafe)

The Bridge: Native Android SMS Manager

How it Works: The system constantly monitors the network. If the phone sees the walkie-talkie line (internet) is dead, it instantly changes its strategy. It connects directly to the phone's native SIM card hardware, compresses the GPS data into a text payload, and sends it over standard 2G cellular voice towers, completely bypassing the internet server.

Step 3: The Server Connecting to the Guardians

The Bridge: Firebase Cloud Messaging (FCM) & WebSockets

How it Works: When the server (Telemetry Hub) receives the emergency event, it immediately fires a "Push Notification" (FCM) to the Guardian's phone. This violently wakes the phone up and overrides silent mode, even if the app is closed. Once the Guardian clicks the notification, their app opens a WebSocket (walkie-talkie) back to the server to listen to the live audio stream.

Step 4: The Guardians Connecting to the Command Center

The Bridge: REST APIs (HTTP POST Requests)

How it Works: The Guardian does not talk directly to the Command Center computer, as that would be a massive security and spam risk. Instead, when the Guardian clicks "Confirm Threat" on their screen, their app sends a tiny, secure data packet (via API) back to the Server. The Server logs that the threat is verified and then securely pushes that final, confirmed data to the Command Center's web dashboard for police dispatch.

3. Engineering Justification: Why We Chose This Architecture **Strict Modular Independence & High Fault Tolerance**

Because we utilize this Broadcaster/Listener method, our system is highly secure, scalable, and stable.

If the Command Center dashboard goes offline for maintenance, or if a specific Guardian loses their internet connection, the Mobile App and the Server still work perfectly. No module relies entirely on the downstream module to function. They are simply broadcasting their data independently, ensuring that the core safety net of the system never fully breaks down. This separation of concerns guarantees 100% uptime reliability for the user in danger.